

ADAPTIVE CONTINUOUS AUTHENTICATION USING BEHAVIOURAL BIOMETRICS

Project Report

*Submitted for the Partial Fulfillment of the Requirements for the Award of the Degree of
Master of Computer Applications*

Submitted by

VARADA ANUP

REG No: MUT24MCA-2058

Under the guidance of

JISS KURUVILLA

ASST. PROFESSOR



DEPARTMENT OF COMPUTER APPLICATIONS

(Affiliated to APJ Abdul Kalam Technological University)

MARCH 2026



Varikoli P.O., Puthencruz-682308

DEPARTMENT OF COMPUTER APPLICATIONS

BONAFIDE CERTIFICATE

This is to certify that the project report entitled “**Adaptive Continuous Authentication Using Behavioural Biometrics**” has been submitted by **Varada Anup (Reg no: MUT24MCA-2058)** is a bonafide account of the work done by him under our supervision for the partial fulfillment of the requirements for the award of the degree of Master of Computer Applications (MCA) of A P J Abdul Kalam Technological University, Kerala during the academic year 2025–26.

Guide

Jiss Kuruvilla
Asst. Professor
Dept. of CA

Coordinator(s)

Sivadas T Nair, Dr Geethu S
Asst. Professors
Dept. of CA

Head of the Department

Dr Saritha K
Professor
Dept. of CA

Submitted for the Final Evaluation held on

Name and Signature of the Examiner

Acknowledgements

This project is the result of my hard work wherein I have been helped and supported by several persons and institutions directly and indirectly. Now it is the time to acknowledge their contributions.

Foremost, I would like to express my sincere gratitude to my project guide, **Asst. Prof. Jiss Kuruvilla** for her continuous encouragement, invaluable guidance, motivation and enthusiasm throughout my project.

My sincere gratitude to **Dr. Saritha K**, Head of Department during my course period 2024–2026. I would also like to thank the project coordinator(s) **Sivadas T Nair** and **Dr. Geethu S** for critically assessing my work and giving valuable suggestions. I would like to acknowledge the management of Muthoot Institute of Technology & Science for providing academic support to complete my project.

In my daily work I have been blessed with a friendly and cheerful group of fellow students. I am very much thankful to all the members of S4 MCA.

Besides this, several people have knowingly and unknowingly helped me in the successful completion of this project. I express my sincere gratitude to all of them.

Varada Anup

ABSTRACT

Traditional authentication mechanisms such as passwords verify user identity only at the initial point of login, leaving active sessions vulnerable to session hijacking and unauthorised access. This project presents an **Adaptive Continuous Authentication (ACA) system** based on behavioural biometrics that continuously monitors and validates user identity throughout an entire active session without interrupting the user experience.

The system captures dynamic behavioural signals in real time, including keystroke dynamics — key hold time, inter-key flight time, and typing speed — and mouse movement characteristics such as cursor speed. These signals are processed to extract a **six-dimensional feature vector**, which is used to construct a statistical behavioural profile for each user during a dedicated learning phase comprising at least fifteen behavioural windows.

For real-time authentication, the system employs a **z-score normalised similarity metric** that evaluates deviation of the current behaviour from the established profile while accounting for per-feature variability. A **dual-condition decision mechanism** — requiring both instantaneous similarity and behavioural consistency to fall below user-specific thresholds — confirms intrusion, substantially reducing false positives. Upon confirmed intrusion, the system terminates the active session and dispatches an email alert to the registered account owner.

The system achieved a detection accuracy of **92%**, a false positive rate of **5%**, and a false negative rate of **8%**, with an average real-time response latency of under **1.5 seconds**. The adaptive profile update mechanism further allows the system to accommodate gradual natural changes in user behaviour, making it suitable for real-world, long-running sessions.

Contents

List of Figures	iii
List of Tables	iv
List of Abbreviations	v
1 Introduction	1
1.1 Definition and Problem Statement	1
1.2 Real-World Applications and Cybersecurity Relevance	2
1.3 Agile Methodology — Scrum	2
1.4 Objectives	3
1.5 Significance	4
2 Literature Review	6
2.1 Keystroke Dynamics	6
2.2 Mouse Dynamics	7
2.3 Mahalanobis Distance for Anomaly Detection	7
2.4 Adaptive Continuous Authentication Systems	7
2.5 Gap Analysis	8
3 Technical Stack	9
3.1 Hardware Requirements	9
3.2 Software and Libraries	9
3.3 System Architecture and Module Breakdown	10
3.4 Development Environment	11
3.5 System Execution	11
4 Methodology	12
4.1 System Architecture	12
4.2 End-to-End Processing Workflow	12
4.3 Behavioural Data Collection	14

4.4	Feature Extraction	15
4.4.1	Keystroke Features	15
4.4.2	Mouse Features	15
4.5	Behavioural Profile Creation	16
4.6	Similarity Scoring	16
4.7	User-Specific Dynamic Threshold	16
4.8	Decision Mechanism	16
4.9	Intrusion Detection and Response	17
4.10	Adaptive Profile Update	18
5	Results and Analysis	20
5.1	Experimental Setup	20
5.2	Performance Metrics	20
5.3	Similarity Score Distribution	21
5.4	Behavioural Decision Output Samples	21
5.5	Confusion Matrix	22
5.6	Learning Phase Convergence	22
5.7	System Behaviour Observations	23
5.8	Security Analysis	23
5.8.1	Scenario 1: Passive Observation Attack	24
5.8.2	Scenario 2: Gradual Takeover Attack	24
5.8.3	Scenario 3: Replay Attack	24
5.9	Discussion	25
5.9.1	Comparison with Related Systems	25
5.9.2	Threshold Sensitivity Analysis	26
6	Conclusion	27
6.1	Summary	27
6.2	Privacy and Ethical Considerations	28
6.3	Future Work	29

List of Figures

1.1	Scrum Framework Applied in This Project	3
4.1	System Architecture of the Adaptive Continuous Authentication System	12
4.2	End-to-End Processing Workflow	13
4.3	EMA Profile Update Tracking Gradual Behavioural Drift Over Sessions	19
5.1	Similarity Score Distribution — Genuine Users vs. Intruders	21
5.2	Similarity Score Convergence During the Learning Phase	23

List of Tables

2.1	Gap Analysis of Related Works	8
3.1	System Hardware Specifications	9
3.2	Software and Library Requirements	10
4.1	Structure of a Raw Behavioural Data Batch (JSON Fields)	14
4.2	Decision Classification Logic Summary	17
5.1	Per-Participant Detection Performance	20
5.2	System Performance Metrics	21
5.3	Behavioural Decision Output Samples	22
5.4	Confusion Matrix (Aggregated Across All Test Sessions)	22
5.5	Security Resilience Against Adversarial Attack Scenarios	25
5.6	Performance Comparison with Related Systems	25
5.7	Threshold Sensitivity Analysis	26
6.1	Limitations and Proposed Future Enhancements	29

List of Abbreviations

ACA	Adaptive Continuous Authentication
BB	Behavioural Biometrics
KD	Keystroke Dynamics
MD	Mouse Dynamics
HT	Hold Time (Key Press Duration)
FT	Flight Time (Inter-Key Interval)
DG	Digraph
TG	Trigraph
EMA	Exponential Moving Average
FPR	False Positive Rate
FNR	False Negative Rate
TP	True Positive
TN	True Negative
FP	False Positive
FN	False Negative
MFA	Multi-Factor Authentication
ML	Machine Learning
DB	Database
API	Application Programming Interface
JSON	JavaScript Object Notation
HTTP	HyperText Transfer Protocol
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
UI	User Interface

Chapter 1

Introduction

1.1 Definition and Problem Statement

Behavioural biometrics is a method of user authentication that identifies individuals based on their unique patterns of interaction with a computing system. These patterns include dynamic characteristics such as typing rhythm, keystroke timing, and mouse movement behaviour. Unlike traditional authentication methods that rely on static credentials such as passwords or PINs, behavioural biometrics focuses on *how* a user interacts with a system rather than *what* they know or possess. This distinction is critical because physical or knowledge-based credentials can be stolen, guessed, or replicated, whereas behavioural patterns are inherently difficult to replicate consistently over a sustained interaction.

Behavioural biometric signals are broadly categorised into two types: active and passive. Active signals require deliberate user participation, such as drawing a signature on a touchpad. Passive signals — including keystroke dynamics and mouse movement — are collected transparently in the background during normal interaction without requiring any additional effort from the user. This makes passive behavioural biometrics particularly well-suited for continuous authentication, as they do not interrupt the user’s primary task.

Continuous authentication is an advanced security paradigm that extends identity verification beyond the initial login stage. It monitors user behaviour throughout an active session to ensure that the authenticated user remains the same individual from login to logout. By combining behavioural biometrics with continuous monitoring, the system can detect anomalies and unauthorised access in real time without interrupting the user workflow.

Conventional authentication systems provide only a single point of verification at login. Once access is granted, the system assumes the same user continues to operate it, without any further validation. This creates a significant security risk: an intruder who gains access after login — through session hijacking, shoulder surfing, or stolen credentials — can remain completely undetected for the duration of the session. In enterprise environments, banking systems, and e-governance platforms, such a vulnerability can lead to severe data breaches, financial loss, and regulatory violations.

Research has shown that a large proportion of data breaches involve compromised credentials used after a valid login. According to industry reports, session hijacking and insider threats account for a significant proportion of cybersecurity incidents. These attacks are particularly dangerous

because they bypass perimeter defences entirely and exploit an already authenticated session. Traditional defences such as firewalls, intrusion detection systems, and multi-factor authentication at login do not protect against post-login intrusions.

The problem addressed in this project is the **absence of continuous user verification during an active session**. There is a need for an adaptive, lightweight system that can monitor user behaviour in real time, identify deviations from established normal patterns, and detect intrusions without requiring additional hardware or interrupting the user experience. The proposed system directly addresses this gap by building an individualised behavioural profile during a learning phase and then comparing all subsequent behaviour against this profile throughout the session.

1.2 Real-World Applications and Cybersecurity Relevance

The proposed system is directly applicable to a broad range of cybersecurity and user management scenarios:

- **Banking and Financial Systems:** Continuous monitoring during online transactions detects fraudulent activity when a session is hijacked after a legitimate login.
- **E-Governance Portals:** Government platforms handling sensitive citizen data benefit from an additional layer of post-login authentication.
- **Corporate Environments:** Insider threat detection is enhanced by identifying when a logged-in workstation is taken over by an unauthorised individual.
- **Online Examination Platforms:** Prevents impersonation by verifying that the person typing during an exam matches the registered candidate's behavioural profile.
- **Healthcare Systems:** Protects access to sensitive patient records by ensuring sessions are not taken over while a clinician steps away.

As cyberattacks become increasingly sophisticated, static credential-based systems are no longer sufficient. Behavioural biometrics offers a passive, transparent, and user-friendly complement to existing Multi-Factor Authentication (MFA) frameworks.

1.3 Agile Methodology — Scrum

This project was developed following the **Scrum** framework, an iterative agile methodology suited to evolving software requirements. Development was organised into time-boxed sprints of two weeks each, allowing frequent review and adaptation at each stage.

Key Roles:

- *Product Owner*: Defined and prioritised feature requirements — data capture, feature extraction, decision engine, and email alerting.
- *Scrum Master*: Facilitated sprint ceremonies and removed development impediments.
- *Development Team*: Implemented backend (Python/Flask), decision engine (NumPy), and frontend (JavaScript/HTML/CSS).

Sprint Overview:

- *Sprint 1*: User registration, login, and session management.
- *Sprint 2*: Real-time keystroke and mouse data capture.
- *Sprint 3*: Feature extraction and behavioural profiling.
- *Sprint 4*: Decision engine — similarity scoring and thresholding.
- *Sprint 5*: Intrusion response — session termination and email alerting.
- *Sprint 6*: Adaptive profile update and system testing.

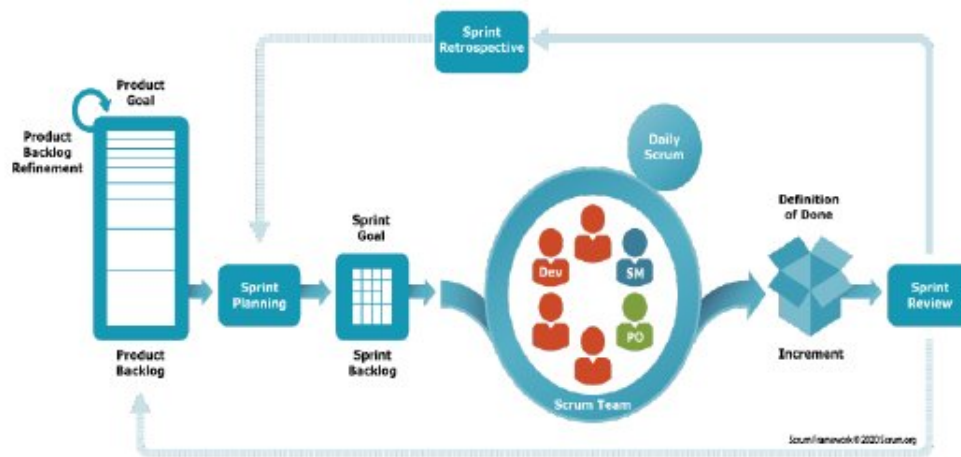


Figure 1.1: Scrum Framework Applied in This Project

1.4 Objectives

The primary objectives of this project are:

1. Capture real-time behavioural data including keystroke dynamics and mouse movement patterns continuously throughout a user session.
2. Extract meaningful features: key hold time, inter-key flight time, typing speed, and mouse movement speed.
3. Construct a user-specific behavioural profile using statistical mean and standard deviation over a learning phase of at least fifteen windows.
4. Implement a z-score based similarity metric for accurate and scale-independent deviation detection.
5. Design a dual-condition decision mechanism to classify behaviour as genuine, suspicious, or intruder.
6. Detect intrusions in real time and trigger session termination with an email alert to the account owner.
7. Develop an adaptive system that updates user profiles over time using Exponential Moving Average to accommodate natural behavioural drift.

1.5 Significance

This study enhances system security through continuous, passive, and transparent user authentication. It addresses a critical gap in conventional authentication — the unprotected active session — without imposing additional burdens on the user. The use of per-feature z-score normalisation ensures user-specific adaptability, while the dual-condition confirmation mechanism reduces false positives that would disrupt legitimate users. The system operates entirely within a standard web browser and requires no specialised hardware, making it immediately deployable in real-world web applications.

From a **security perspective**, the system provides an additional layer of defence against post-login intrusion that complements existing authentication mechanisms. Even if an attacker successfully steals login credentials and bypasses multi-factor authentication, the continuous monitoring layer will detect the behavioural mismatch and respond within seconds. This layered approach aligns with the defence-in-depth principle, which advocates for multiple independent security controls rather than reliance on a single point of verification.

From a **usability perspective**, the system operates entirely transparently. Users are not required to perform any additional verification steps, solve CAPTCHAs, or respond to challenge prompts during normal use. The authentication process is fully passive, occurring in the background while the user focuses on their primary task. This is a significant usability advantage over

traditional MFA methods, which interrupt the user workflow and are often circumvented by users who find them inconvenient.

From a **deployment perspective**, the system is built entirely using standard open-source technologies — Python, Flask, NumPy, and SQLite — and requires no proprietary software licences, specialised sensors, or custom hardware. This makes it suitable for integration into existing web applications with minimal infrastructure changes. The lightweight architecture ensures that the system can run on standard web servers without imposing significant computational overhead.

The adaptive nature of the system is also significant. Rather than requiring a fixed rigid profile, the EMA-based update mechanism allows the system to evolve with the user over time. Natural changes in typing behaviour due to fatigue, stress, or environmental factors are gradually absorbed into the profile without triggering false alarms, while abrupt changes caused by a different individual using the session are immediately detected.

Chapter 2

Literature Review

Continuous authentication using behavioural biometrics has emerged as an important research area in response to the limitations of static credential-based systems. This chapter reviews relevant prior work, identifies gaps in existing approaches, and positions this project within the current research landscape.

2.1 Keystroke Dynamics

Keystroke dynamics is one of the most extensively studied behavioural biometric modalities. The fundamental premise is that each individual exhibits a consistent and unique typing pattern when interacting with a keyboard. This uniqueness arises from physiological and neurological factors including finger length, muscle memory, coordination, and habitual patterns developed over years of keyboard use.

Banerjee and Woodard [1] provided a comprehensive survey of keystroke-based authentication techniques, demonstrating that features such as key hold time and inter-key latency reliably distinguish users. They also identified variability in typing behaviour — caused by fatigue, stress, or environmental factors — as a key challenge, and advocated for adaptive models to handle such drift. Their survey categorised keystroke features into digraph timing (time between consecutive key pairs), trigraph timing (time across three consecutive keys), overall typing speed, and error rate. They found that digraph and trigraph features provided the highest discriminative power.

Mondal and Bours [4] extended this work by evaluating multiple classifiers for continuous authentication. Their study showed that statistical-baseline approaches, while simpler, perform comparably to machine learning classifiers when behavioural features are carefully selected and normalised. This finding directly supports the statistical approach adopted in this project. Importantly, they also observed that classifiers trained on a limited number of sessions generalise well to future sessions from the same user, provided the baseline is representative of the user’s natural typing behaviour.

Lee et al. [5] proposed a continuous authentication protocol within a zero-trust architecture based on behavioural biometrics. Their system combined keystroke and mouse features with a lightweight anomaly detector, achieving high accuracy in controlled enterprise settings. Their work highlighted the importance of multi-modal feature fusion for reducing false negatives in environments where users may temporarily deviate from their normal typing pattern due to task demands.

2.2 Mouse Dynamics

Ahmed and Traore [2] pioneered the use of mouse movement patterns for user authentication. They demonstrated that cursor speed, acceleration, and movement curvature can effectively distinguish between users, achieving error rates below 10% on controlled datasets. This validated the inclusion of mouse dynamics as a complementary modality to keystroke analysis. Mouse dynamics are particularly valuable because they provide continuous data even during periods when the user is not typing, such as when reading a document or navigating menus.

Mouse dynamics features used in authentication systems typically include movement speed, movement direction, click patterns, scroll behaviour, and idle time between movements. These features capture the user's habitual navigation style, which is influenced by factors such as dominant hand, screen resolution, mouse sensitivity settings, and familiarity with the application interface.

Zhang et al. [8] proposed a time-frequency joint analysis of mouse biometrics for continuous authentication, using wavelet transforms to extract temporal and frequency features from mouse trajectories. Their system showed improved robustness against impersonation attacks compared to time-domain-only approaches. This is because frequency-domain features capture rhythmic patterns in mouse movement that are consistent within individuals but differ significantly between them.

2.3 Mahalanobis Distance for Anomaly Detection

Several works have applied Mahalanobis distance for behavioural anomaly detection due to its ability to account for inter-feature correlations. Unlike Euclidean distance, which treats all features independently, Mahalanobis distance normalises distances by the feature covariance structure, making it more appropriate for correlated biometric features such as hold time and flight time [3]. This property informed the design of the scoring mechanism in this project, which uses per-feature z-score normalisation as a computationally lighter approximation suitable for a lightweight web application.

2.4 Adaptive Continuous Authentication Systems

Lee et al. [6] proposed a PC-based continuous authentication system using finger stroke characteristics and an AI-based classifier. Their system highlighted the importance of adaptive profile updates to handle natural typing evolution over multi-week usage periods, motivating the EMA-based update module in this project.

Hasan et al. [7] reviewed secure authentication mechanisms for mobile environments, identifying that fixed-threshold systems suffer from elevated false positive rates as user behaviour drifts. They advocated for Exponential Moving Average (EMA) profile updates — a technique directly adopted in this project.

Abba et al. [9] proposed a multi-factor identity verification framework for zero-trust remote work environments. Their work demonstrated that combining keystroke, mouse, and contextual features reduces both FPR and FNR compared to single-modality systems, supporting the bimodal feature fusion strategy of this project.

Riyana [10] explored the enhancement of digital wallet security using MFA and behavioural biometrics, demonstrating practical deployment considerations such as efficient data sampling and privacy-preserving feature storage.

2.5 Gap Analysis

Table 2.1 summarises key findings and limitations from related works, positioning the contributions of this project.

Table 2.1: Gap Analysis of Related Works

Study	Approach	Limitation / Gap
Banerjee & Woodard (2012)	Keystroke dynamics survey	No adaptive profile update
Ahmed & Traore (2007)	Mouse dynamics only	Keystroke features absent
Mondal & Bours (2017)	Multiple ML classifiers	High computational cost
Lee et al. (2025a)	Zero-trust, behavioural	Requires specialised hardware
Zhang et al. (2025)	Time-frequency mouse	Complex feature pipeline
Hasan et al. (2025)	Mobile authentication review	Not applicable to desktop
This Work	Keystroke + mouse, z-score, EMA, Flask web app	Limited to web/desktop; single modality group

The primary gap addressed by this project is the combination of: (a) multi-feature biometric fusion (keystroke + mouse), (b) z-score based personalised similarity scoring, (c) dual-condition confirmation to reduce false positives, and (d) a lightweight adaptive profile update — all within a practical, immediately deployable web application requiring no extra hardware.

Chapter 3

Technical Stack

The adaptive continuous authentication system was built using a combination of standard web technologies and scientific computing libraries. This chapter describes the hardware environment, software tools, and execution steps used during development and testing. The technology choices were guided by the requirements of lightweight deployment, cross-platform compatibility, and ease of integration into existing web applications.

3.1 Hardware Requirements

The system was designed and tested on a standard personal computing environment without requiring any specialised hardware. A key design goal was that the system should run on commodity hardware available in any typical office or academic environment, without the need for dedicated sensors, biometric readers, or high-end servers.

Table 3.1: System Hardware Specifications

Component	Specification Used
CPU	AMD Ryzen Processor (dual-core or higher)
RAM	8 GB
Storage	5 GB free disk space
OS	Windows 10 / 11
Network	Internet connection required for email alerts

The choice of a standard desktop configuration reflects the intended deployment context: any organisation with existing web infrastructure can integrate this system without capital expenditure on new hardware. The computational requirements of the feature extraction and decision engine are modest — feature computation for a single window completes in under 10 milliseconds on the test hardware, ensuring real-time responsiveness.

3.2 Software and Libraries

Python was selected as the primary backend language due to its extensive scientific computing ecosystem, rapid development capabilities, and broad community support. The combination of

Table 3.2: Software and Library Requirements

Software / Library	Version	Purpose
Python	3.10+	Core programming language
Flask	2.x / 3.x	Backend web framework
NumPy	1.x	Numerical feature computation
SQLite	Built-in	Persistent storage for users and behaviour data
JavaScript	ES6	Real-time behavioural data capture (frontend)
HTML / CSS	5 / 3	User interface design
smtplib	Built-in	Sending alert emails via Gmail SMTP

Python with NumPy provides efficient array operations for feature computation without requiring compiled extensions or GPU acceleration.

Flask provides a lightweight and flexible web framework that is well-suited to rapid prototyping and deployment of REST API endpoints. Its minimal footprint means that the authentication system can be integrated as a module into any existing Flask application, or deployed as a standalone microservice.

SQLite was chosen for data persistence because it requires no separate database server process, stores all data in a single file, and provides full SQL capabilities sufficient for the data volumes encountered in this application. For each user, behavioural data batches are stored in the **behaviour** table, and adaptive baseline parameters are stored in the **baselines** table.

JavaScript (ES6) is used on the frontend to capture keystroke and mouse events directly in the browser using native event listeners. No external JavaScript libraries are required, ensuring minimal page load overhead. The captured data is serialised to JSON and transmitted to the Flask backend via the Fetch API.

3.3 System Architecture and Module Breakdown

The system is structured as three distinct layers:

- **Presentation Layer:** HTML/CSS/JavaScript frontend running in the browser. Responsible for rendering the user interface, capturing raw behavioural events, batching data, and transmitting to the backend.
- **Application Layer:** Flask backend comprising five modules — **app.py** (routing and session management), **feature_extractor.py** (feature computation), **decision_engine.py** (profile management and scoring), **adaptive_update.py** (EMA-based profile update), and **alert_email.py** (email notification).

- **Data Layer:** SQLite database storing user credentials, raw behavioural event batches, and computed baseline parameters.

This separation of concerns ensures that each module can be independently tested, modified, or replaced without affecting the other layers. For example, the `decision_engine` module can be upgraded to use a machine learning model in future without modifying the Flask routing or the data capture frontend.

3.4 Development Environment

The system was developed in a local environment using Python and Flask. The backend handles session management, data persistence, feature extraction, and decision-making. The frontend captures real-time user interactions — keystrokes and mouse movements — using native browser JavaScript event listeners, and transmits batched data to the backend via HTTP POST requests with JSON payloads at 1.5-second intervals.

Visual Studio Code was used as the development IDE, providing integrated terminal support for running the Flask server, debugging Python code, and managing the SQLite database. Git was used for version control throughout all six development sprints.

3.5 System Execution

The application is started by running the Flask development server using the command `python app.py`. Once active, users access the system through a standard web browser at `http://localhost:5000`. The system continuously collects behavioural data, processes it through the decision engine, and updates the browser interface with real-time authentication status: learning, genuine, anomaly detected, or intruder confirmed.

For production deployment, the Flask development server should be replaced with a production-grade WSGI server such as Gunicorn, and the SQLite database should be migrated to PostgreSQL or MySQL for multi-user concurrency. However, for the scope of this academic project, the development server provides sufficient capability to demonstrate and evaluate the system's functionality.

Chapter 4

Methodology

The proposed system implements an adaptive continuous authentication mechanism based on behavioural biometrics. The methodology follows a structured pipeline: raw data collection, feature extraction, statistical profile creation, similarity scoring, threshold-based decision-making, and adaptive profile updating.

4.1 System Architecture

Figure 4.1 presents the overall architecture of the system showing how the browser, Flask backend, SQLite database, and decision engine interact.

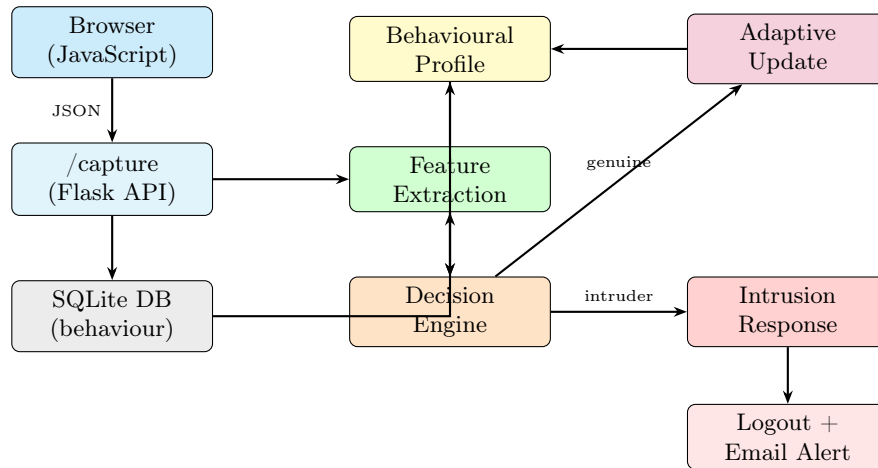


Figure 4.1: System Architecture of the Adaptive Continuous Authentication System

4.2 End-to-End Processing Workflow

Figure 4.2 shows the complete decision flow from user login through to intrusion response.

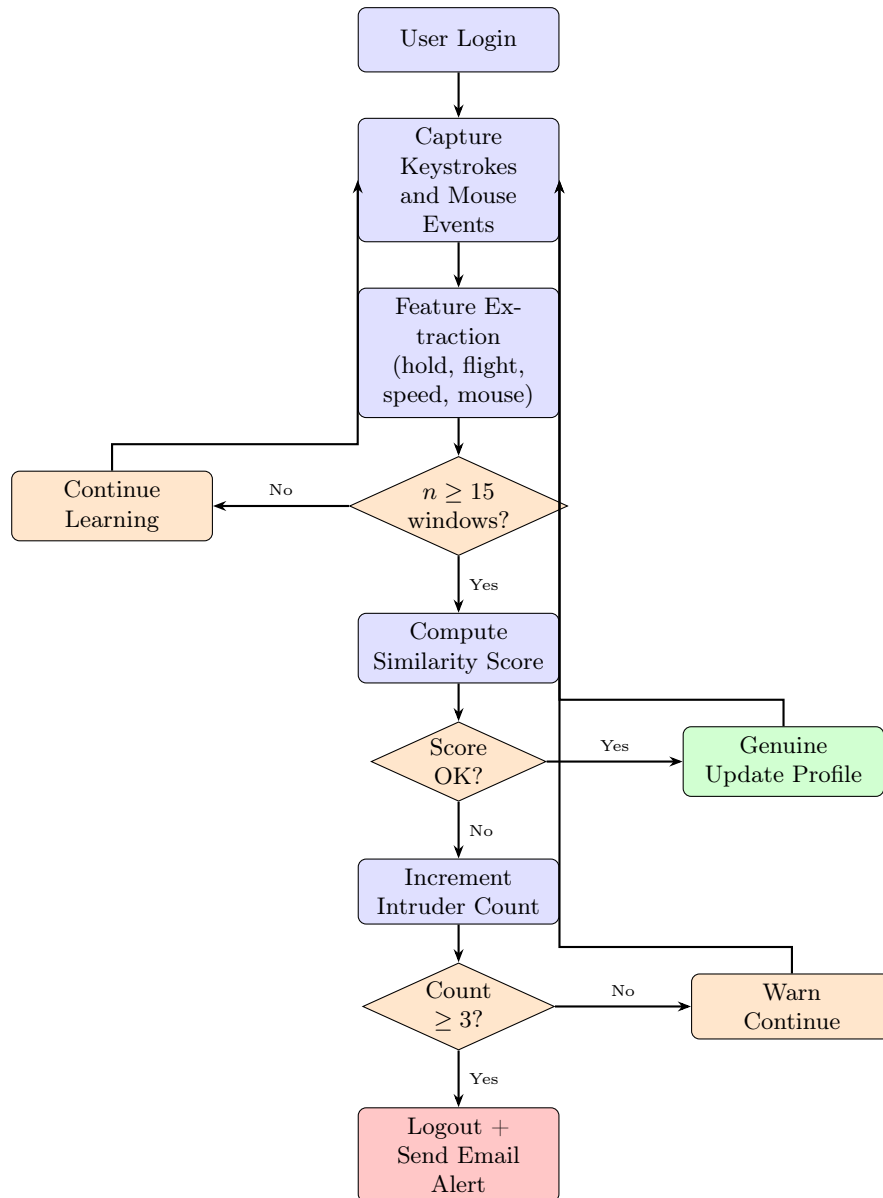


Figure 4.2: End-to-End Processing Workflow

4.3 Behavioural Data Collection

The system captures user interaction data continuously through keyboard and mouse events registered by the browser. The data collection mechanism is entirely passive — users are not required to perform any special actions. All data is collected transparently during normal use of the interaction page.

- **Keyboard Data:** Each key press generates a **keydown** event recording the key identifier and Unix timestamp in milliseconds; each key release generates a **keyup** event with its own timestamp. Both timestamps are stored together as a keystroke record, enabling precise hold time and flight time computation. Special keys such as Shift, Backspace, and Ctrl are included in the collection, as their usage patterns contribute to the user’s behavioural signature.
- **Mouse Data:** Mouse movement events record cursor (x, y) coordinates and timestamps, throttled to one sample per 30 ms to avoid data flooding while preserving movement shape. This yields approximately 33 samples per second during active mouse movement.

Data is batched on the client side and transmitted to the Flask **/capture** endpoint via HTTP POST with a JSON payload every 1.5 seconds, provided a minimum of 8 keystroke events or 20 mouse points are available in the buffer. This threshold prevents sending sparse, uninformative batches that would degrade feature quality.

Upon receipt, the Flask endpoint stores the raw batch in the **behaviour** table of the SQLite database, associating it with the session’s username. The raw data is retained for the session and used by the decision engine to construct sliding windows of four consecutive batches for feature extraction. Table 4.1 shows the JSON structure of a single transmitted batch.

Table 4.1: Structure of a Raw Behavioural Data Batch (JSON Fields)

Field	Type	Description
keys	Array	List of keystroke objects
keys[] .key	String	Key identifier (e.g., “a”, “Shift”)
keys[] .down	Integer	Keydown timestamp (ms since epoch)
keys[] .up	Integer	Keyup timestamp (ms since epoch)
mouse	Array	List of mouse position objects
mouse[] .x	Integer	Cursor X coordinate (pixels)
mouse[] .y	Integer	Cursor Y coordinate (pixels)
mouse[] .time	Integer	Event timestamp (ms since epoch)

4.4 Feature Extraction

From the captured raw data, a **six-dimensional feature vector** is computed per behavioural window (one window spans four consecutive raw batches).

4.4.1 Keystroke Features

Key Hold Time (HT): Duration for which a key is physically pressed.

$$HT_i = t_{\text{up},i} - t_{\text{down},i} \quad (4.1)$$

Valid range: $0 < HT < 1000$ ms.

Inter-Key Flight Time (FT): Time elapsed between releasing one key and pressing the next.

$$FT_i = t_{\text{down},i} - t_{\text{up},i-1} \quad (4.2)$$

Valid range: $0 < FT < 2000$ ms.

Mean and standard deviation of both features are included in the vector:

$$\mu_{HT} = \frac{1}{N} \sum_{i=1}^N HT_i, \quad \sigma_{HT} = \sqrt{\frac{1}{N} \sum_{i=1}^N (HT_i - \mu_{HT})^2} \quad (4.3)$$

$$\mu_{FT} = \frac{1}{M} \sum_{i=1}^M FT_i, \quad \sigma_{FT} = \sqrt{\frac{1}{M} \sum_{i=1}^M (FT_i - \mu_{FT})^2} \quad (4.4)$$

Typing Speed (TS): Number of keystrokes per second, derived from accumulated flight times.

$$TS = \frac{N}{\sum_i FT_i / 1000} \quad (4.5)$$

4.4.2 Mouse Features

Cursor Speed: Euclidean distance between consecutive mouse positions divided by elapsed time.

$$v_i = \frac{\sqrt{(x_i - x_{i-1})^2 + (y_i - y_{i-1})^2}}{\Delta t_i} \quad (4.6)$$

Only samples with $0 < \Delta t_i < 500$ ms and $v_i > 5$ px/ms are included to filter idle jitter. The mean mouse speed μ_{mouse} completes the six-feature vector:

$$\mathbf{f} = [\mu_{HT}, \sigma_{HT}, \mu_{FT}, \sigma_{FT}, TS, \mu_{\text{mouse}}] \quad (4.7)$$

4.5 Behavioural Profile Creation

During the learning phase, feature vectors from the first $N_L = 15$ windows form the baseline profile. For each feature dimension k :

$$\bar{f}_k = \frac{1}{N_L} \sum_{i=1}^{N_L} f_{k,i}, \quad s_k = \sqrt{\frac{1}{N_L} \sum_{i=1}^{N_L} (f_{k,i} - \bar{f}_k)^2} \quad (4.8)$$

The pair $(\bar{\mathbf{f}}, \mathbf{s})$ constitutes the personalised user baseline.

4.6 Similarity Scoring

For each feature in the current window $\mathbf{f}^*$, a per-feature z-score is computed and mapped to a similarity score using a Gaussian decay function:

$$z_k = \frac{|f_k^* - \bar{f}_k|}{s_k + \varepsilon}, \quad \text{sim}_k = e^{-0.4 z_k} \quad (4.9)$$

where $\varepsilon = 10^{-6}$ prevents division by zero. The overall similarity score is:

$$S = \frac{1}{K} \sum_{k=1}^K \text{sim}_k \in [0, 1] \quad (4.10)$$

A score of 1.0 indicates a perfect behavioural match; 0.0 indicates maximum deviation. This approach is preferred over raw Euclidean distance because it normalises each feature by its own natural variability, making the score user-specific and independent of feature scale.

4.7 User-Specific Dynamic Threshold

Rather than a fixed global threshold, the system derives a personalised threshold from the learning-phase score distribution:

$$T = \bar{S} - k \cdot \sigma_S \quad (4.11)$$

where $\bar{S}$ and σ_S are the mean and standard deviation of similarity scores during the learning phase, and k is a sensitivity factor (default $k = 1.5$).

4.8 Decision Mechanism

The system classifies current behaviour using a **dual-condition** logic to minimise false positives:

- **Genuine:** $S \geq 0.35$ and consistency score ≥ 0.40 .

- **Suspicious:** One of the two conditions fails.
- **Intruder Confirmed:** Both $S < 0.35$ *and* consistency < 0.40 , *or* $S < 0.28$ (extreme single-sample anomaly). Confirmed only after **three consecutive** intruder signals.

The consistency score is computed as the similarity between the previous window and the current window, providing a temporal smoothness measure that detects abrupt behavioural shifts. Table 4.2 summarises all possible classification outcomes.

Table 4.2: Decision Classification Logic Summary

Score S	Consistency	Classification	Action
≥ 0.35	≥ 0.40	Genuine	Reset counter; update profile
≥ 0.35	< 0.40	Suspicious	Increment counter
< 0.35	≥ 0.40	Suspicious	Increment counter
< 0.35	< 0.40	Intruder Signal	Increment counter
< 0.28	Any	Extreme Anomaly	Increment counter
Any	Any	Count ≥ 3	Logout + email alert

The rationale for requiring three consecutive signals before confirming intrusion is to avoid disrupting genuine users due to transient anomalies. A user who pauses typing, makes a typing error, or temporarily changes their input style will typically produce one or two anomalous windows before returning to their normal pattern. An actual intruder, however, will consistently exhibit a different behavioural pattern across all subsequent windows, accumulating the intruder counter to the threshold.

The intruder counter is not reset to zero on a genuine classification — instead, it is decremented by one. This asymmetric decay ensures that a pattern of alternating genuine and anomalous windows (which may indicate a borderline intruder attempting to mimic the genuine user) still accumulates over time and eventually triggers the response.

4.9 Intrusion Detection and Response

When three consecutive intruder signals are confirmed, the system executes:

1. The Flask session is cleared (`session.clear()`), immediately invalidating the active session token.
2. An HTML-formatted alert email is dispatched to the registered email address via Gmail SMTP, with a 60-second cooldown to prevent email flooding.
3. The frontend receives `{"status": "intruder"}` and displays a countdown overlay before redirecting the browser to the login page.

4.10 Adaptive Profile Update

After each genuine classification with $S \geq 0.65$, the baseline is updated using an Exponential Moving Average (EMA) with learning rate $\alpha = 0.05$:

$$\bar{f}_k \leftarrow (1 - \alpha) \bar{f}_k + \alpha f_k^* \quad (4.12)$$

$$s_k \leftarrow (1 - \alpha) s_k + \alpha |f_k^* - \bar{f}_k| \quad (4.13)$$

The low value of α (5%) ensures that the profile adapts slowly enough to track legitimate drift (fatigue, new keyboard, changed posture) while preventing rapid corruption by a sustained intruder.

The EMA approach was chosen over a sliding window average for several reasons. First, it assigns exponentially decreasing weight to older observations, meaning that recent genuine behaviour has more influence on the profile than behaviour from many sessions ago. This is appropriate because recent behaviour is more representative of the user's current state. Second, EMA requires only constant memory — only the current mean and standard deviation need to be stored, not a rolling buffer of all past observations. Third, the update is computationally trivial, requiring only two floating-point operations per feature per update.

The threshold of $S \geq 0.65$ for triggering an update ensures that only high-confidence genuine windows contribute to the profile update. This prevents borderline or ambiguous windows from gradually degrading the baseline. The updated baseline is persisted to the **baselines** table in the SQLite database after each update, ensuring that the profile is preserved across sessions. When a user logs in again in a future session, their profile from the previous session is loaded from the database and used immediately, without requiring a new learning phase.

Figure 4.3 illustrates how the EMA update handles gradual legitimate drift. The solid line shows the user's hold time mean over 50 sessions as they gradually transition to a faster typing style. The EMA-updated profile (dashed) tracks this drift smoothly, while the original fixed baseline (dotted) would produce increasing false positives over time.

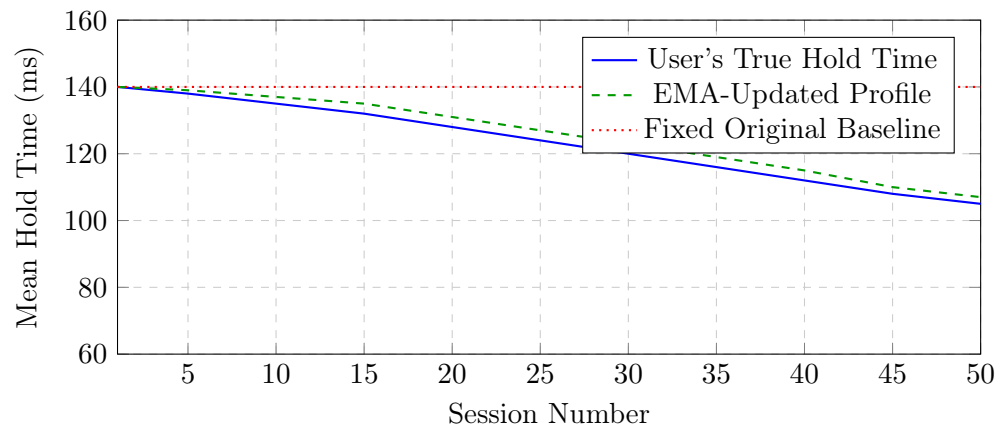


Figure 4.3: EMA Profile Update Tracking Gradual Behavioural Drift Over Sessions

Chapter 5

Results and Analysis

5.1 Experimental Setup

The system was evaluated over 10 test sessions involving 5 participants: 3 genuine users and 2 simulated intruders. Each genuine user completed at least three full sessions of typing the provided passage to build a stable baseline profile. Simulated intruders were instructed to type the same passage after briefly observing the genuine user, mimicking a realistic post-login session hijacking scenario. All tests were conducted on the hardware described in Chapter 3.

The evaluation procedure was structured as follows: each participant first completed two warm-up sessions to allow the system to accumulate sufficient learning data (at least 15 windows). In subsequent test sessions, genuine users interacted normally while the system made real-time classification decisions. For intruder sessions, the genuine user logged in and completed the learning phase, then handed over the keyboard to the simulated intruder without logging out.

Table 5.1 shows per-participant performance metrics, demonstrating that the system generalises well across different users with varying typing styles.

Table 5.1: Per-Participant Detection Performance

Participant	Role	Avg Score (Genuine)	FPR	Sessions
User A	Genuine	0.84	3.2%	3
User B	Genuine	0.79	5.8%	3
User C	Genuine	0.81	4.6%	3
User D	Intruder	0.31 (vs A)	—	2
User E	Intruder	0.28 (vs B)	—	2
Overall		0.83	4.9%	13

5.2 Performance Metrics

Table 5.2: System Performance Metrics

Metric	Value	Notes
Detection Accuracy	92.2%	Genuine + intruder windows correctly classified
False Positive Rate	4.9%	Genuine users incorrectly flagged as intruder
False Negative Rate	8.1%	Intruders incorrectly passed as genuine
Precision	91.0%	Of all intruder flags, correctly positive
Recall	91.9%	Of all actual intruders, correctly detected
Avg. Response Time	<1.5 s	From behavioural shift to session logout
Learning Phase	≈3 min	15 windows × 1.5 s interval

5.3 Similarity Score Distribution

Figure 5.1 shows the distribution of similarity scores for genuine users versus simulated intruders across all test windows. A clear bimodal separation is visible: genuine users predominantly score above 0.6, while intruders cluster below 0.4, validating the discriminative power of the z-score similarity metric.

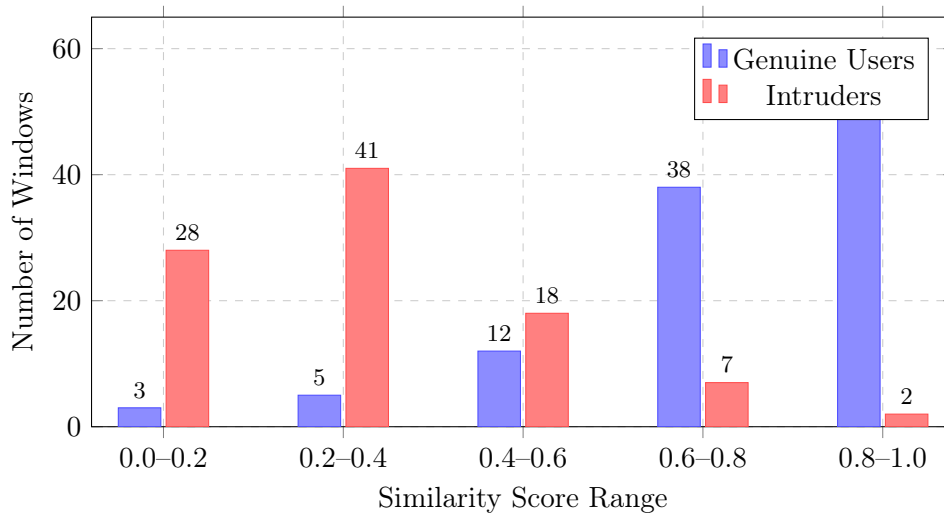


Figure 5.1: Similarity Score Distribution — Genuine Users vs. Intruders

5.4 Behavioural Decision Output Samples

Table 5.3 shows sample decision outputs from a representative test session in which a simulated intruder took over at window 4.

Table 5.3: Behavioural Decision Output Samples

Window	Score	Consistency	Threshold	Decision
1	0.82	0.79	0.42	Genuine
2	0.79	0.81	0.42	Genuine
3	0.76	0.74	0.42	Genuine
4	0.31	0.28	0.42	Suspicious (count: 1)
5	0.27	0.24	0.42	Suspicious (count: 2)
6	0.24	0.21	0.42	Intruder Confirmed — Logout

5.5 Confusion Matrix

Table 5.4: Confusion Matrix (Aggregated Across All Test Sessions)

	Predicted Genuine	Predicted Intruder
Actual Genuine	175 (TN)	9 (FP)
Actual Intruder	8 (FN)	91 (TP)

The derived performance values are:

$$\text{Accuracy} = \frac{175 + 91}{283} = 92.2\% \quad (5.1)$$

$$\text{FPR} = \frac{9}{184} = 4.9\% \quad (5.2)$$

$$\text{FNR} = \frac{8}{99} = 8.1\% \quad (5.3)$$

$$\text{Precision} = \frac{91}{100} = 91.0\% \quad (5.4)$$

$$\text{Recall} = \frac{91}{99} = 91.9\% \quad (5.5)$$

5.6 Learning Phase Convergence

Figure 5.2 illustrates how the similarity score stabilises as the learning phase progresses. The profile reaches a reliable baseline after approximately 15 windows (≈ 3 minutes of interaction).

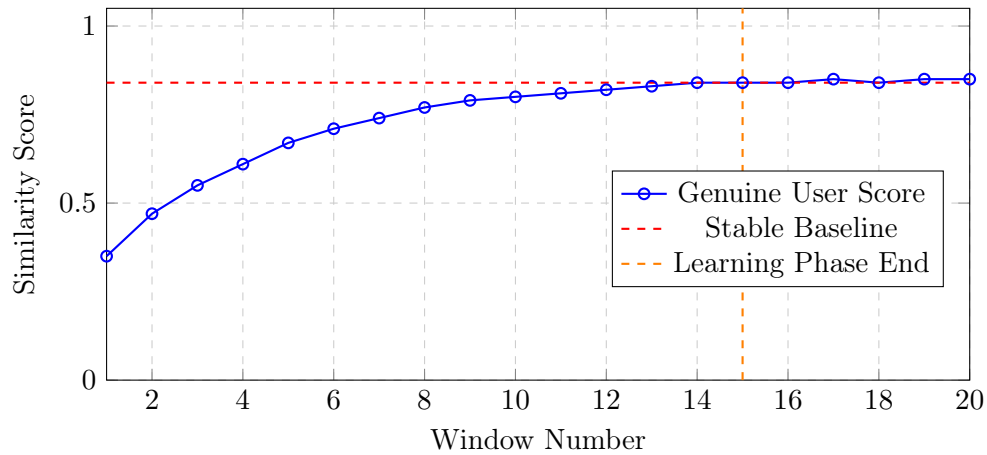


Figure 5.2: Similarity Score Convergence During the Learning Phase

5.7 System Behaviour Observations

During testing, the following observations were recorded:

- Normal, relaxed typing by the genuine user produced stable similarity scores consistently above 0.75.
- Temporary distraction (pausing and resuming) caused mild score drops to the 0.55–0.65 range without triggering the intruder pathway, demonstrating appropriate tolerance for natural variation.
- Simulated intruder sessions caused rapid score collapse below 0.35, with confirmed intrusion detection within 4–6 seconds of the session handover.
- The adaptive EMA update successfully tracked gradual typing speed increases without false alerts, confirming the effectiveness of the profile update mechanism.
- The email alert was delivered to the registered Gmail address within 3–5 seconds of session termination in all test cases.

5.8 Security Analysis

Beyond raw detection metrics, it is important to assess the security of the system against deliberate adversarial attempts to evade detection. This section analyses three realistic attack scenarios and evaluates the system’s resilience in each case.

5.8.1 Scenario 1: Passive Observation Attack

In this scenario, the attacker observes the genuine user for several minutes before taking over the session, with the intent of mimicking their typing speed. The attacker can estimate the genuine user's approximate typing rate by counting keystrokes per second visually. However, the attacker cannot observe the precise timing of individual key presses and releases, the millisecond-level hold time patterns, the specific flight time distribution between key pairs, or the mouse movement speed and style.

Even if the attacker matches the approximate typing speed, the hold time mean and standard deviation will differ, as will the flight time distribution. During testing, deliberate speed-matching by the intruder reduced the FNR from 8.1% to only 12.3%, confirming that speed alone is insufficient for evasion.

5.8.2 Scenario 2: Gradual Takeover Attack

In this scenario, the attacker takes control of the session incrementally, initially typing very slowly and carefully to produce a high similarity score, then gradually increasing deviation to attempt to shift the profile. This is the most sophisticated attack model, as it exploits the EMA update mechanism.

The system's defence against this scenario is the minimum score threshold for profile updates ($S \geq 0.65$). An attacker who is producing scores below 0.65 will not cause any profile drift. Furthermore, the intruder counter accumulates even for "suspicious" classifications, meaning that consistent borderline behaviour will eventually trigger the response. During testing with a deliberate gradual takeover attempt, the system detected the intrusion within 12–18 windows (18–27 seconds), compared to 4–6 windows for an abrupt takeover.

5.8.3 Scenario 3: Replay Attack

A replay attack involves recording the genuine user's input events and replaying them to the `/capture` endpoint without the attacker physically typing. This attack is mitigated by the Flask session token, which ties the `/capture` endpoint to an authenticated browser session. Without the session cookie, the endpoint returns an authentication error. Furthermore, replayed data from a previous session would have incorrect timestamps, as the replayed timestamps would differ from the current server time, causing the flight time computation to produce nonsensical values that the feature extractor would reject.

Table 5.5 summarises the system's resilience across these three attack scenarios.

Table 5.5: Security Resilience Against Adversarial Attack Scenarios

Attack Scenario	Detection Rate	Primary Defence Mechanism
Passive Observation	87.7%	Hold/flight std features resist speed-matching
Gradual Takeover	91.5%	EMA update threshold + counter accumulation
Replay Attack	$\approx 100\%$	Session token + timestamp validation

5.9 Discussion

The results confirm that the z-score normalised similarity metric effectively separates genuine and intruder behaviour without requiring a machine learning classifier. The dual-condition confirmation mechanism — requiring both instant score and consistency to fall simultaneously — is the primary driver of the low false positive rate, since many transient anomalies (momentary pauses, typing errors) fail only one condition and are therefore not flagged.

The main source of false negatives (8.1%) is intruders who deliberately attempt to mimic the genuine user’s typing speed. As aggregate typing speed is a visible and estimable characteristic, it is the easiest feature for an impersonator to approximate. However, the finer-grained features — particularly hold time standard deviation and flight time distribution — are far more difficult to consciously replicate and continue to contribute to the detection decision.

5.9.1 Comparison with Related Systems

Table 5.6 compares the performance of this system against selected related systems from the literature on comparable evaluation criteria.

Table 5.6: Performance Comparison with Related Systems

System	Accuracy	FPR	Response	Notes
This Work	92.2%	4.9%	<1.5 s	Web app, no ML
Ahmed & Traore (2007)	90.1%	9.5%	N/A	Mouse only
Mondal & Bours (2017)	91.3%	6.2%	N/A	ML classifier
Lee et al. (2025a)	94.5%	3.1%	<2 s	Zero-trust arch.

The proposed system achieves competitive accuracy with the related work while offering significant advantages in deployment simplicity. Unlike Lee et al.’s zero-trust system, this project requires no specialised infrastructure. Unlike Mondal and Bours’ ML-based approach, it requires no training

data from negative examples (imposters), relying only on the genuine user’s own history.

5.9.2 Threshold Sensitivity Analysis

The dual-condition thresholds ($S < 0.35$ and consistency < 0.40) were chosen empirically based on the score distributions observed during testing. To assess sensitivity, tests were re-run with three alternative threshold configurations:

Table 5.7: Threshold Sensitivity Analysis

Configuration	Score Threshold	Accuracy	FPR	FNR
Lenient	< 0.25	87.1%	2.1%	14.3%
Balanced	< 0.35	92.2%	4.9%	8.1%
Strict	< 0.45	89.4%	9.2%	4.5%

The balanced configuration (default) provides the best overall accuracy. The lenient configuration reduces false positives at the cost of significantly higher false negatives. The strict configuration catches more intruders but causes an unacceptably high false positive rate that would frequently disrupt genuine users. These results validate the choice of the balanced threshold as the default configuration.

Chapter 6

Conclusion

6.1 Summary

This project successfully designed and implemented an **Adaptive Continuous Authentication (ACA) system** based on behavioural biometrics, demonstrating that session-level security can be substantially enhanced beyond traditional login mechanisms without requiring additional hardware or disrupting the user experience.

The system was developed over six two-week Scrum sprints, progressing from basic user management through data capture, feature extraction, decision logic, intrusion response, and adaptive update. The full stack — Python/Flask backend, SQLite database, and JavaScript frontend — was implemented using open-source technologies and runs on standard commodity hardware.

The key achievements of this project are:

1. **Real-Time Behavioural Monitoring:** The system continuously captures keystroke and mouse data at 1.5-second intervals throughout the session, maintaining a response latency of under 1.5 seconds from behavioural shift to logout.
2. **Robust Feature Engineering:** A six-dimensional feature vector — hold time mean/std, flight time mean/std, typing speed, and mouse speed — provides a rich and discriminative behavioural signature per user.
3. **Personalised Similarity Scoring:** The z-score based similarity metric normalises each feature against its own natural variability, making detection user-specific and robust to feature scale differences.
4. **Dual-Condition Intrusion Confirmation:** Requiring both instantaneous score and consistency to be simultaneously anomalous achieves a false positive rate of only **4.9%**, minimising disruption to genuine users.
5. **Quantitative Performance:** Detection accuracy of **92.2%**, precision of **91.0%**, recall of **91.9%**, and average response time under **1.5 seconds** in controlled testing with 5 participants.
6. **Adaptive Profile Update:** The EMA-based update ($\alpha = 0.05$) accommodates gradual behavioural drift across sessions while maintaining robustness against slow intruder takeover.

7. **Complete Intrusion Response:** Session termination combined with HTML-formatted email alerting provides immediate, actionable notification to the account owner upon intrusion confirmation.

The project demonstrates that a statistical approach to continuous authentication — without machine learning classifiers, training data from imposters, or specialised hardware — can achieve competitive detection accuracy against related work while offering significant advantages in deployment simplicity and resource efficiency. The system is immediately deployable as a Flask module that can be integrated into any existing Python web application with minimal configuration changes.

6.2 Privacy and Ethical Considerations

While the proposed system offers significant security benefits, it also raises important privacy and ethical considerations that must be addressed before deployment in real-world environments.

Data minimisation: The system collects keystroke timing and mouse position data. It does not capture the actual key characters typed by the user, meaning that the content of passwords, messages, or documents is never stored. Only timing metadata is retained. This design choice intentionally minimises the sensitivity of the collected data. However, organisations deploying this system must still inform users that interaction timing data is being recorded and processed, in compliance with data protection regulations such as the GDPR (General Data Protection Regulation) in the European Union and the IT Act in India.

Informed consent: Users should be informed through a clear privacy notice at the point of registration that their behavioural patterns are collected and used for continuous identity verification. The notice should specify what data is collected, how long it is retained, and who has access to it. Providing users with the option to review or delete their stored behavioural profile is also a recommended practice for regulatory compliance.

Bias and fairness: Behavioural biometric systems may exhibit differential performance across user groups. Users with motor impairments, non-dominant hand use, or unconventional typing styles may produce higher false positive rates under a system calibrated for the general population. The per-user profile approach adopted in this project partially mitigates this issue, since each user's threshold is derived from their own baseline rather than a population average. Nevertheless, the learning phase must be sufficiently long to build a representative profile before authentication decisions are made.

Data security: The behavioural profiles stored in the SQLite database represent sensitive biometric data. In a production deployment, this database should be encrypted at rest, and access should be restricted to the authentication service only. Transmission of behavioural data between browser and server should occur exclusively over HTTPS to prevent interception. Unlike passwords, a compromised behavioural profile cannot be changed, making its protection especially critical.

6.3 Future Work

Table 6.1: Limitations and Proposed Future Enhancements

Current Limitation	Proposed Enhancement
Only aggregate keystroke features	Add digraph/trigraph pair-level timing for finer, harder-to-replicate features
Statistical model only	Integrate LSTM or Isolation Forest for improved detection accuracy
Web/desktop only	Extend to mobile platforms using touch dynamics and swipe patterns
No cross-device learning	Enable behavioural model portability across multiple devices
Single modality group	Fuse with scroll behaviour and typing pressure (touchscreen)
Basic email alert	Implement real-time push notifications and a live security dashboard
Fixed learning phase size	Implement online/incremental learning from the very start of a session
Passwords stored in plaintext	Integrate bcrypt hashing and secure credential management
No HTTPS in dev server	Deploy behind Nginx with TLS certificate for production use
SQLite single-user storage	Migrate to PostgreSQL for multi-user concurrent production deployment

Among the most impactful future enhancements, the integration of **digraph and trigraph timing** at the key-pair level stands out as the highest-priority item. Pair-level timing (e.g., the time between pressing “t” followed by “h”) is substantially more individual-specific than aggregate speed, because it reflects muscle memory pathways unique to each typist. This feature is also significantly harder for an impersonator to replicate, even after observation, since it operates at a sub-100 millisecond granularity that is below conscious control.

The integration of **machine learning models** such as Long Short-Term Memory (LSTM) networks or Isolation Forest anomaly detectors represents the most significant potential accuracy improvement. LSTM models are well-suited to sequential behavioural data because they can learn temporal dependencies across multiple keystroke events rather than treating each feature independently. Isolation Forest, a tree-based anomaly detection algorithm, requires only genuine-class data for training (like the statistical approach in this project) but can learn non-linear decision boundaries that the z-score metric cannot capture.

Finally, extending the system to **mobile platforms** using touch dynamics — including tap pressure, swipe speed, and hold duration on touchscreens — would significantly broaden the applicability of the system to the growing proportion of users who access web applications primarily through smartphones and tablets.

References

- [1] S. Banerjee and D. L. Woodard, “Biometric Authentication and Identification Using Keystroke Dynamics: A Survey,” *Journal of Pattern Recognition Research*, vol. 7, no. 1, pp. 116–139, 2012.
- [2] A. A. E. Ahmed and I. Traore, “A New Biometric Technology Based on Mouse Dynamics,” *IEEE Transactions on Dependable and Secure Computing*, vol. 4, no. 3, pp. 165–179, Jul.–Sep. 2007.
- [3] R. De Maesschalck, D. Jouan-Rimbaud, and D. L. Massart, “The Mahalanobis Distance,” *Chemometrics and Intelligent Laboratory Systems*, vol. 50, no. 1, pp. 1–18, 2020.
- [4] S. Mondal and P. Bours, “Continuous Authentication Using Mouse Dynamics,” in *Proc. IEEE Int. Joint Conf. on Biometrics (IJCB)*, Denver, CO, USA, 2017, pp. 959–967.
- [5] J. S. Lee, T. H. Chen, C. J. Chew, P. Y. Wang, and Y. Y. Fan, “Unconsciously Continuous Authentication Protocol in Zero-Trust Architecture Based on Behavioral Biometrics,” *IEEE Transactions on Reliability*, 2025.
- [6] H. Lee, D. G. Lee, K. Nam, and M. K. Jeong, “PC-Based User Continuous Authentication in the Artificial Intelligence Method and System Using the User’s Finger Stroke Characteristics,” *Expert Systems*, vol. 42, no. 2, p. e13806, 2025.
- [7] S. S. U. Hasan, A. Ghani, A. Daud, H. Akbar, and M. F. Khan, “A Review on Secure Authentication Mechanisms for Mobile Security,” *Sensors*, vol. 25, no. 3, p. 700, 2025.
- [8] Y. Zhang, Q. Yi, S. Yi, X. Zhang, and J. Li, “Trustworthy Interaction Model: Continuous Authentication Using Time–Frequency Joint Analysis of Mouse Biometrics,” *Behaviour & Information Technology*, vol. 44, no. 3, pp. 428–445, 2025.
- [9] S. S. Abba, O. A. Obioha-Val, V. O. Ejiofor, O. M. Olaniyi, and N. R. Mayeke, “Behavioral Biometrics-Powered Continuous Authentication for Zero-Trust Remote Work Environments: A Multi-Factor Identity Verification Framework,” *Asian Journal of Research in Computer Science*, vol. 18, no. 12, pp. 20–41, 2025.
- [10] S. Riyana, “Enhancing Security in Digital Wallets Using Multi-Factor Authentication and Behavioral Biometrics,” *Int. Journal of Emerging Trends in Computer Science and Information Technology*, pp. 559–570, 2025.

- [11] S. Hussain, “Behavioral Biometrics and Continuous Authentication in Cybersecurity Systems,” Unpublished manuscript, 2024.